Drips: RepoDriver

Security Review

Cantina Managed review by:
R0bert, Lead Security Researcher
High Byte, Security Researcher

March 17, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 High Risk . . . 4
3.1.1 Upgrade can strand legacy Gelato balances . . . 4
3.2 Medium Risk . . . 4
3.2.1 Reused Repo paths can seize accounts . . . 4
3.2.2 Pre-migration Lit payloads can override migrated accounts . . . 4
3.2.3 Codeberg null suffix can revoke short accounts . . . 5
3.2.4 GitLab user/group reuse can seize accounts . . . 5
3.2.5 Lit oracle chain-string padding collision lets attacker revoke RepoDriver ownership and flip RepoDeadlineDriver payout . . . 6
3.3 Low Risk . . . 6
3.3.1 Hugging Face YAML can revoke valid owners . . . 6
3.3.2 Oracle fetches have no response size limit . . . 6
3.3.3 GitHub case aliases split one identity . . . 7
3.3.4 Consider longer timeout & retries . . . 7
3.4 Informational . . . 7
3.4.1 Reused domain names can seize accounts . . . 7
3.4.2 `gitLabUser` claims break under DPoP . . . 7
3.4.3 Zero-owner revocation triggered by HTTP states . . . 8
3.4.4 Website names can rewrite fetch targets . . . 8
3.4.5 Token-based claim modes expand trust boundary . . . 8
3.4.6 General recommendation: use typescript . . . 9
3.4.7 Gitlab uri encoding is incomplete . . . 9
3.4.8 Permissionless emitAccountId can spam events . . . 9
3.4.9 Short name encoding collides on null suffix . . . 9
3.4.10 Oracle CLI fails on current node . . . 10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Mar 4th to Mar 9th the Cantina team conducted a review of contracts on commit hash 6d7150a5. The team identified a total of **20** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	0	1
Medium Risk	5	3	2
Low Risk	4	1	3
Gas Optimizations	0	0	0
Informational	10	4	6
Total	**20**	**8**	**12**

2.1 Scope

The security review had the following components in scope for contracts on commit hash 6d7150a5:

```
├── oracle
│   └── litAction.js
└── src
    └── RepoDriver.sol
```

3 Findings

3.1 High Risk

3.1.1 Upgrade can strand legacy Gelato balances

Severity: High Risk

Context: RepoDriver.sol#L16

Description: The previous `RepoDriver` implementation held native-token balances for Gelato fee payments. Users could deposit funds, keep a personal balance on the proxy, and withdraw any unused remainder through `withdrawUserFunds`. The Lit-based implementation removes that interface, while the upgrade proposal switches the proxy straight to the new implementation.

If any deployment still has nonzero `userFunds` or shared `commonFunds` when the upgrade executes, the ETH stays in the proxy but the user-controlled withdrawal path disappears immediately. Nothing is stolen during the upgrade, but the funds become stranded behind governance or a follow-up rescue implementation instead of remaining withdrawable by the users who deposited them.

This makes the migration unsafe unless every upgraded deployment can prove that those balances are already zero, or the upgrade is staged so users can exit first.

Recommendation: Do not execute the direct Gelato-to-Lit upgrade unless legacy fee balances are confirmed to be zero. The safer options are to enforce a pre-upgrade invariant on `userFundsTotal` and `commonFunds`, or to ship an intermediate implementation that disables new Gelato activity while keeping withdrawals open.

If a direct upgrade must stay available, the proposal needs an on-chain migration or rescue path that preserves user-level withdrawal rights instead of converting recovery into an admin-only process.

Drips: Acknowledged. As of now across 4 deployments there are no tokens locked as `userFunds`. There is a slim chance that in the window before the update somebody does actually lock funds, but in that case it's easier to reimburse them manually. Implementing a withdrawal path would increase complexity and introduce new risks while probably never being genuinely usable by anybody. We will monitor this.

Cantina Managed: Acknowledged.

3.2 Medium Risk

3.2.1 Reused Repo paths can seize accounts

Severity: Medium Risk

Context: RepoDriver.sol#L16

Description: `RepoDriver` keys GitHub and GitLab repositories by raw path bytes, not by a stable repository ID. If an old path is later reclaimed after a username, namespace, transfer, or path change, the new path controller can obtain a fresh oracle signature for the same `(sourceId, name)` pair and take over the existing Drips account. For example, if Alice changes her GitHub username from `alice` to `alicehq`, Bob can later register `alice`, recreate `alice/stream-payments` and overwrite the owner of the original Drips account.

Recommendation: Key repository accounts by stable upstream repository IDs instead of mutable path strings. If migration is not possible, treat path changes as explicit account migrations rather than as identity continuity.

Drips: Acknowledged. Documented in commit f08b4d6d.

Cantina Managed: Acknowledged.

3.2.2 Pre-migration Lit payloads can override migrated accounts

Severity: Medium Risk

Context: RepoDriver.sol#L187-L215

Description: Before the Gelato-to-Lit upgrade, accounts only store an owner address. After the upgrade, the same storage is interpreted as `AccountOwner { address owner; uint32 timestamp; }`, which

leaves migrated legacy accounts with their existing owner but `timestamp == 0`. Any Lit payload that was signed before the upgrade and carries a positive timestamp can therefore become the first accepted `updateOwnerByLit` write on that account, even if the off-chain ownership signal has already changed again. In practice, an outdated pre-migration Lit claim can overwrite the migrated owner until someone submits a newer Lit update.

Recommendation: Gate the first Lit update for migrated legacy accounts so pre-migration payloads cannot be used after the upgrade. For example, require the first accepted Lit timestamp on migrated accounts to be at or after the migration activation time.

Drips: Fixed in commit f08b4d6d.

Cantina Managed: Fix verified.

3.2.3 Codeberg null suffix can revoke short accounts

Severity: Medium Risk

Context: RepoDriver.sol#L123-L125

Description: Short names collide under `bytes27(name)` if they differ only by a trailing null byte. For the `codeberg` repository source, the null-suffixed variant resolves to a meaningful `404`, which the oracle treats as revocation and signs as a zero-owner update. An attacker can therefore query the colliding null-suffixed name for a short Codeberg repo and repeatedly clear the owner of the original account without ever becoming the new owner.

Recommendation: Use a length-preserving encoding or always hash `name` and reject null bytes before the oracle signs them.

Drips: Fixed in f08b4d6. It's verified on the oracle side.

Cantina Managed: Fix verified.

3.2.4 GitLab user/group reuse can seize accounts

Severity: Medium Risk

Context: litAction.js#L38-L43

Description: The Lit oracle assigns `sourceId = 8` to both `gitLabUser` and `gitLabGroup` and `RepoDriver` derives the controlled account only from `(sourceId, name)`. As a result, a GitLab user account and a GitLab group with the same canonical path are treated as the same on-chain identity even though they are different upstream entity types and are proven through different oracle flows.

This becomes exploitable because `updateOwnerByLit` accepts any oracle-signed payload whose `timestamp` is strictly newer than the last stored timestamp, but does not enforce any expiry or freshness window. The `gitLabUser` path signs with the personal access token `created_at` value, so while an attacker still controls a GitLab username they can mint a fresh token, obtain a valid signature for that `(sourceId, name)` and keep it indefinitely. If the GitLab path is later reassigned to a group with the same name, the cached signature still targets the same `RepoDriver` account and can be replayed to restore the attacker as the on-chain owner.

Once the stale signature is submitted, the attacker regains control over the Drips account until a newer GitLab-derived update is posted. During that window they can call owner-gated functions such as `collect`, `give`, `setStreams`, `setSplits` and `emitAccountMetadata`, which is sufficient to drain funds or reconfigure future flows.

Recommendation: Do not let GitLab users and GitLab groups share the same identity space. At minimum, assign them different `sourceId` values so a user path and a group path cannot resolve to the same `accountId`. For a durable fix, bind GitLab identities to stable upstream principal IDs instead of mutable path strings.

Drips: Acknowledged. Won't fix, documented in commit f08b4d6d. Users and groups intentionally share the same namespace, so the collision risk between a user and a group is no different from the existing collision risk between two users or two groups. A separate source ID would therefore not reduce the underlying risk.

Cantina Managed: Acknowledged.

3.2.5 Lit oracle chain-string padding collision lets attacker revoke RepoDriver ownership and flip RepoDeadlineDriver payout

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The Lit Action selects repo ownership claims by exact chain string equality but signs the EIP-712 payload using `bytes32(chain)` via `formatBytes32String`. Because bytes32-string encoding is right-zero-padded, distinct strings differing only by trailing NUL bytes (e.g., "sepolia" vs "sepolia\u0000") encode to the same on-chain `bytes32`. An attacker can request a NUL-padded chain string so the oracle finds no matching claim and signs `owner = address(0)` for a `bytes32` value that still matches `RepoDriver.chain`. Submitting this signature to `RepoDriver.updateOwnerByLit` revokes an otherwise-claimed repo account and flips `RepoDeadlineDriver.collectAndGive` to the refund branch after the deadline, depriving intended recipients of funds.

Recommendation: In `contracts/oracle/litAction.js`, canonicalize requested chains to a `bytes32` form first and use that canonical value consistently for de-duplication (prevent multiple strings mapping to the same `bytes32`), claim selection (compare by `bytes32` value, not raw string), and signing.

Reject chain strings containing `\u0000` and other non-printable characters.

Consider changing `FUNDING.json` to store chain identifiers as `bytes32` (or a collision-resistant hash) instead of free-form strings.

Drips: Fixed in commit f08b4d6d.

Cantina Managed: Fix verified.

3.3 Low Risk

3.3.1 Hugging Face YAML can revoke valid owners

Severity: Low Risk

Context: litAction.js#L286-L298

Description: The Hugging Face lookup reads `ownedBy` from YAML frontmatter, and an unquoted `0x...` value is parsed as a number instead of a string. Address parsing then fails, but the lookup is still treated as meaningful, so the oracle can sign a fresh zero-owner update. A small README formatting mistake can therefore clear an existing owner until a corrected claim is posted.

Recommendation: Require `ownedBy` to be a string before address validation, or parse frontmatter with a schema that preserves plain scalars as strings. Malformed claims should be rejected, not treated as revocation.

Drips: Acknowledged. Documented in commit f08b4d6d.

Cantina Managed: Acknowledged.

3.3.2 Oracle fetches have no response size limit

Severity: Low Risk

Context: litAction.js#L23-L25, litAction.js#L153-L156, litAction.js#L383-L400

Description: The fetch layer has no response-size limit and buffers the same body twice with `arrayBuffer()` and `text()`. An attacker-controlled website can return a very large response and force the Lit action to spend extra memory, time and fee budget on content that should have been rejected early. This makes claim refreshes less reliable and shifts avoidable work onto the caller.

Recommendation: Enforce a small response-size cap and stop reading once it is exceeded. The fetch layer should read the response body only once, instead of first consuming `response.clone().arrayBuffer()` and then consuming `response.text()` again on the original response.

Drips: Acknowledged. Won't fix as there are other ways for the user to harm themselves and the potential damage is low.

Cantina Managed: Acknowledged.

3.3.3 GitHub case aliases split one identity

Severity: Low Risk

Context: litAction.js#L14-L16, litAction.js#L35-L37, litAction.js#L153-L164, litAction.js#L180-L193, RepoDriver.sol#L117-L144

Description: The GitHub sources sign the path exactly as the caller typed it. GitHub itself treats `octocat/spoon-knife` and `OctoCat/SPOON-KNIFE` as the same repository, but `calcAccountId` treats those strings as different bytes and derives different Drips accounts from them. The same GitHub user or repo can therefore end up split across multiple accounts depending only on casing.

Recommendation: Normalize GitHub names to one canonical form before they are signed and before `calcAccountId` is derived. The safest approach is to fetch the canonical owner and repo names from GitHub and always use that exact returned path, rather than trusting caller input. Do not silently apply that change to existing accounts, because the canonical path may hash to a different account ID than the mixed-case path users already rely on. Instead, introduce a new source version for canonicalized GitHub identities or provide a migration that moves ownership and configuration from the old mixed-case account to the new canonical one.

Drips: Acknowledged. Documented in commit f08b4d6d.

Cantina Managed: Acknowledged.

3.3.4 Consider longer timeout & retries

Severity: Low Risk

Context: litAction.js#L383-L400

Description: During peak traffic, load times may take longer than 5 seconds, or temporarily reject or drop the response.

Recommendation: Use a longer timeout (default for timeout is 30 seconds) and add retries for retryable failures. A simple approach is a thirty second timeout with three attempts.

Drips: Fixed in commit f08b4d6d.

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 Reused domain names can seize accounts

Severity: Informational

Context: RepoDriver.sol#L16

Description: The `website` source keys accounts by the raw `host[/path]` string and signs whatever controller currently serves `https://<name>/FUNDING.json`. If that domain expires, is transferred, or the hosted path changes hands, the new controller can take over the same Drips account. In practice, the account belongs to whoever controls that website today, not to a stable long-term identity.

Recommendation: Consider documenting this behaviour. Otherwise, do not treat website accounts as persistent identities. If control of the domain or path changes, create a new account and migrate users and configuration to it instead of reusing the old one.

Drips: Fixed in commit f08b4d6d by documenting this behaviour.

Cantina Managed: Fix verified.

3.4.2 `gitLabUser` claims break under DPoP

Severity: Informational

Context: litAction.js#L195-L219

Description: The `gitLabUser` flow authenticates with only the `PRIVATE-TOKEN` header. GitLab accounts that enable DPoP require an additional `DPoP` proof header, so this source stops working for those users.

Funds are not stolen, but legitimate owners may be unable to refresh or recover their claim through the GitLab user path.

Recommendation: Add DPoP support to the GitLab user flow or mark DPoP-enabled accounts unsupported for this source. If unsupported, document that limitation clearly in the user flow.

Drips: Fixed in commit f08b4d6d by documenting this behaviour.

Cantina Managed: Fix verified.

3.4.3 Zero-owner revocation triggered by HTTP states

Severity: Informational

Context: litAction.js#L102-L106

Description: The oracle treats `401`, `403`, and `404` as meaningful absence and defaults missing claims to the zero address when revocation is enabled. Temporary auth failures, rate limits, or provider-side policy changes can therefore produce a signed zero-owner update without proving that ownership was intentionally removed. Anyone can submit that payload and clear the account until a newer update arrives.

Recommendation: Consider requiring a stronger evidence before signing revocation. Ambiguous HTTP failures should trigger retry or no-op, not immediate owner clearing.

Drips: Acknowledged. Partially fixed in commit f08b4d6d. Added retrials when the response is not 2xx, 401, 403 or 404, which covers network failures and cases like HTTP 429 and 5xx errors that may go away on retrials.

Cantina Managed: Fix verified. The fix does improve the transient-failure as it adds retries for non-2xx responses that are not 401/403/404, so 429, 5xx and network failures no longer immediately feed into revocation. But the core behavior remains: in `litAction`, 401, 403 and 404 are still treated as "meaningful", that flag still becomes `revokeUnclaimed` and missing claims still default to `AddressZero`. So 401/403 can still produce a signed zero-owner update.

3.4.4 Website names can rewrite fetch targets

Severity: Informational

Context: litAction.js#L23-L25, litAction.js#L153-L156, litAction.js#L383-L386

Description: The `website` source concatenates the raw `name` string into a URL without validation. Inputs containing userinfo, ports, queries, or fragments can therefore make the oracle fetch a different target than the signed website string suggests. This weakens the identity binding for website accounts and may also reach unexpected destinations.

Recommendation: Parse and validate `name` as strict `host[/path]`. Reject all other URL components and rebuild the request from validated parts.

Drips: Acknowledged.

Cantina Managed: Acknowledged.

3.4.5 Token-based claim modes expand trust boundary

Severity: Informational

Context: litAction.js#L38-L40, litAction.js#L47-L49, litAction.js#L50-L52, litAction.js#L195-L197, litAction.js#L235-L239, litAction.js#L259-L263

Description: The `gitLabUser`, `codebergUser` and `huggingFaceUser` flows require users to supply reusable third-party API tokens to the Lit execution environment. That expands the trust boundary beyond the user and the upstream provider. If the Lit environment or its assumptions fail, the leaked token can usually be used directly against the user's account.

This should be fine given the Lit documentation, as requests are processed inside TEEs and the request contents are not exposed to operators during normal operation but is definitely a third party risk to consider:

- `https://developer.litprotocol.com/learning-lit/communicating-with-lit-nodes`

- https://developer.litprotocol.com/node-ops/requirements

Recommendation: Prefer proof flows that do not require reusable credentials. If these modes remain, disclose the trust tradeoff clearly and recommend the narrowest scope and shortest lifetime tokens possible.

Drips: Acknowledged. The minimum token privileges are documented.

Cantina Managed: Acknowledged.

3.4.6 General recommendation: use typescript

Severity: Informational

Context: litAction.js#L383-L400

Description/Recommendation: In the current codebase there are lots of validations and typechecks that can be done by leveraging typescript & libraries for example Axios to get typed structured data.

Drips: Acknowledged.

Cantina Managed: Acknowledged.

3.4.7 Gitlab uri encoding is incomplete

Severity: Informational

Context: litAction.js#L221-L233

Description: In Gitlab group claim the `nameUri` is built by simply encoding slashes in `name`, but that is incomplete. If the `name` contains `?` or `#` symbols the remainder of the url will be improperly moved from path to query or hash respectively.

Recommendation: Consider using `encodeURIComponent`.

Drips: Acknowledged.

Cantina Managed: Acknowledged.

3.4.8 Permissionless emitAccountId can spam events

Severity: Informational

Context: RepoDriver.sol#L151-L154

Description: `emitAccountId` is permissionless and emits `AccountIdSeen` for arbitrary caller-supplied inputs. Anyone can therefore spam the event stream with fake discovery signals. This does not grant ownership, but indexers and monitoring should treat the event as untrusted.

Recommendation: Treat `AccountIdSeen` only as a hint unless it is paired with a trusted ownership update.

Drips: Acknowledged.

Cantina Managed: Acknowledged.

3.4.9 Short name encoding collides on null suffix

Severity: Informational

Context: RepoDriver.sol#L123-L125

Description: Short names are encoded with `bytes27(name)`, which right-pads with zeros and discards explicit trailing null bytes. Two distinct short byte strings can therefore collide if they differ only by a null suffix. Whether that is exploitable depends on the source, but the collision exists in the encoding itself.

Recommendation: Reject names with null bytes or switch to a length-preserving encoding. Always hashing `name` would remove the short-name ambiguity entirely.

Drips: Fixed in commit f08b4d6d. It's verified on the oracle side.

Cantina Managed: Fix verified.

3.4.10 Oracle CLI fails on current node

Severity: Informational

Context: index.js#L6

Description: The manual oracle tooling is documented as a normal operator workflow, but the CLI entrypoint does not start on the current local Node runtime. The entry script imports `mkdtempDisposable` from `node:fs/promises` at module load time and later uses it to create the temporary storage directory for Lit auth state. On the current environment, that export is not available, so Node aborts before any subcommand logic runs.

The failure is easy to reproduce with the exact command the documentation tells operators to use for deployment inspection.

```
$ npm run getDeployment

> oracle@0.0.1 getDeployment
> node index.js getDeployment

file:///.../oracle/index.js:6
import { mkdtempDisposable } from "node:fs/promises";
         ^^^^^^^^^^^^^^^^^
SyntaxError: The requested module 'node:fs/promises' does not provide an export named
↪  'mkdtempDisposable'
```

In local testing this happened on Node `v24.2.0`. The same top-level import failure also prevents other documented commands from running, because the process dies before it ever dispatches to `getDeployment`, `deposit`, `queryByName`, or `queryByToken`. This is not limited to the one path that actually needs temporary storage. As written, the whole CLI is coupled to a runtime-specific API at import time.

That makes the operator story worse than it first appears. The README tells users to run `npm run getDeployment` to print the oracle IPFS hash and the derived signing addresses for each Lit network. That is exactly the kind of command a deployer, reviewer, or responder needs when checking which signer a `RepoDriver` instance should trust. Instead, the tool crashes before it prints anything useful. The same goes for the manual query commands that are supposed to help users fetch a fresh signed ownership payload. If someone is trying to verify a deployment, rotate infrastructure, or recover from a stale owner record, the documented local tooling is unavailable until they first debug the Node compatibility problem.

This is not an on-chain security bug, but it is a real operational break. The repository does not declare a strict supported Node version and the documentation only says that `npm` is needed. A user following the documented setup can therefore end up with a nonfunctional CLI even though installation succeeds and the failure only appears at runtime.

Recommendation: Stop depending on `mkdtempDisposable` and `await using` in the CLI entrypoint. Use `mkdtemp` to create the temporary directory, pass the resulting path into the Lit auth storage setup and clean it up in a `try` / `finally` block with an explicit removal step. That keeps the behavior the same without tying the tool to a narrow runtime feature.

If the project intentionally requires a specific Node release, declare it explicitly in `package.json` and in the README so operators fail early with a clear version requirement rather than a runtime import crash. A simple regression check in CI should run the documented commands, especially `npm run getDeployment`, on the supported Node version range so this kind of breakage is caught before release.

Drips: Fixed in commit f08b4d6d.

Cantina Managed: Fix verified.